Certordle v0.0.1 PRD / Technical Spec

1. Product Summary

Product name: Certordle
Version: 0.0.1
Product type: Certification study game / daily quiz
Inspired by: Wordle-style daily guessing games and Doctordle-style prompt-based guessing experiences
Primary goal: Help certification candidates practice high-value certification concepts through short, repeatable, clue-based puzzles.

Certordle is a daily certification study game where users receive a short prompt or scenario and must guess the correct certification-related concept, service, resource, tool, or architectural pattern. The first proof of concept will focus on cloud and AI developer certifications, especially Microsoft Azure certifications and Anthropic/Claude developer topics.

Unlike traditional practice exams, Certordle is designed to reinforce recognition, differentiation, and recall of similar concepts. The experience should help users answer questions like:

- Which service best fits this scenario?
- How is this concept different from a similar service?
- What resource/tool/pattern does this clue describe?
- Which technologies are commonly confused on certification exams?

The v0.0.1 POC will prioritize correctness, low cost, fast feedback, and content maintainability over advanced personalization.

---

2. Problem Statement

Certification learners often struggle not because they have never seen a concept, but because they confuse similar technologies under exam pressure.

Examples:

- Azure AI Search vs. Cosmos DB
- Azure Monitor vs. Application Insights vs. Log Analytics
- Event Grid vs. Event Hubs vs. Service Bus
- Managed Identity vs. SAS token vs. Key Vault secret
- Azure OpenAI vs. Azure AI Foundry vs. Azure AI Search
- Claude Tool Use vs. MCP vs. Claude Code Skills

Most cert prep materials are either:

1. Long-form documentation

2. Flashcards
3. Full practice exams
4. Video courses
5. Static multiple-choice quizzes

Certordle fills a smaller daily-practice niche: one prompt, one answer, fast repetition, smart feedback, and an explanation after completion.

---

3. Product Goals

3.1 Primary Goals

- Create a low-cost POC for certification-based daily guessing puzzles.
- Support multiple certification tracks over time.
- Use Git as the source of truth for v0.0.1 content.
- Avoid runtime AI/LLM usage for guess interpretation or feedback.
- Provide useful deterministic feedback for wrong guesses.
- Save only completed quiz results and aggregate quiz statistics.
- Make the product feel educational, lightweight, and habit-forming.

3.2 Secondary Goals

- Make content easy to review, version, and update.
- Allow future migration from Git-backed content to database-backed production content.
- Support daily puzzles, archived puzzles, and track-specific puzzle packs.
- Support account-based streaks and cross-device progress later.
- Allow AI-assisted content generation offline during the admin/content creation workflow, not during gameplay.

---

4. Non-Goals for v0.0.1

The following are intentionally out of scope for v0.0.1:

- Runtime LLM interpretation of user guesses.
- Full adaptive learning paths.
- Paid subscriptions.
- Admin dashboard.
- User-generated puzzles.
- Real exam question recreation.
- Live syncing with Microsoft or Anthropic exam updates.
- Deep analytics dashboards.
- Saving every individual guess permanently.
- Complex spaced repetition system.
- Mobile app.
- Multiplayer or competitive play.

---

5. Target Users

5.1 Primary User

Certification candidates studying for technical certifications who want quick daily review.

Example users:

- Azure Developer Associate candidates
- Azure AI Engineer Associate candidates
- Cloud developers preparing for interviews
- Engineers learning AI tooling
- Developers learning Claude, MCP, and agentic workflows

5.2 Secondary User

Technical learners who are not actively pursuing a certification but want to sharpen platform knowledge.

Example users:

- Software developers learning Azure
- AI engineers learning RAG architecture
- Consultants preparing for client-facing conversations
- Developers learning Anthropic/Claude tooling

---

6. Initial Certification Tracks

v0.0.1 should support the concept of multiple tracks, but the POC does not need to launch with all of them fully populated.

6.1 Recommended Initial Track

AZ-204: Azure Developer Associate

Reason:

- Highly relevant to developer workflows.
- Contains many puzzle-friendly services.
- Strong overlap with real-world architecture decisions.
- Rich set of commonly confused services.

6.2 Candidate Tracks

Track	Priority	Notes
AZ-204	High	Best first POC candidate
AI-102	High	Strong fit for AI/RAG/service-differentiation puzzles
AZ-900	Medium	Easier audience, simpler concepts
Anthropic / Claude Developer	Medium	Useful once certification scope is clearer
Mixed Azure Review	Medium	Could combine AZ-204, AI-102, AZ-900
Agentic AI / MCP Pack	Medium	Good non-cert learning pack

---

7. Core Gameplay

7.1 Basic Flow

1. User selects a certification track.
2. User sees the daily puzzle prompt.
3. User enters guesses using a constrained concept autocomplete.
4. Each guess returns deterministic feedback.
5. User continues until they solve the puzzle or run out of guesses.
6. On completion, user sees:
7. Correct answer
8. Explanation
9. Related concepts
10. Source/study reference
11. Result summary
12. Shareable result
13. The app stores final result and aggregate statistics.

7.2 Puzzle Format

Each puzzle has:

- Certification track
- Domain/objective
- Difficulty
- One-line or short scenario prompt
- Correct answer concept
- Accepted aliases
- Optional custom feedback for common wrong guesses

- Fallback feedback based on relationships/categories
- Explanation shown after completion
- Optional source/reference URL
- Tags

7.3 Guessing Rules

- User should not submit arbitrary free text directly.
- User types into an autocomplete/search input.
- Guess must resolve to a known concept ID.
- Aliases should resolve to canonical concepts.
- Misspellings may be supported later through fuzzy matching.
- Invalid guesses should not count against the user unless intentionally designed otherwise.

Example invalid guess response:

> I do not recognize that as a valid concept in this certification set. Try selecting one of the suggested concepts.

7.4 Guess Limit

Recommended v0.0.1 default:

- 6 guesses per puzzle

This matches the familiar Wordle-style constraint while allowing enough room for technical concept confusion.

7.5 Completion States

A puzzle can end in one of two states:

1. **Solved**

2. User guessed the correct concept within the allowed number of guesses.

3. **Failed**

4. User used all allowed guesses without selecting the correct answer.

Both states should show the final explanation.

---

8. Example Puzzle

8.1 Azure AI Search Example

Track: AI-102
Domain: Knowledge mining / Retrieval-Augmented Generation
Difficulty: Medium
Answer: Azure AI Search

Prompt:

> I create searchable indexes over enterprise content and can support keyword, semantic, and vector retrieval for grounded generative AI apps.

Accepted aliases:

- Azure AI Search
- Cognitive Search
- Azure Cognitive Search
- Azure Search
- AI Search

Common wrong guess feedback:

Guess	Feedback
Cosmos DB	Not quite. Cosmos DB stores operational application data, while Azure AI Search creates indexes for keyword, semantic, and vector retrieval.
Azure OpenAI	Close. Azure OpenAI can generate responses, but Azure AI Search retrieves the grounding data used in many RAG applications.
Document Intelligence	Related. Document Intelligence extracts text and structure from documents; Azure AI Search indexes content for retrieval.
Blob Storage	Related. Blob Storage can hold source files, but Azure AI Search indexes and retrieves their content.

Reveal explanation:

> Azure AI Search is used to create searchable indexes over structured and unstructured content. It can support keyword search, semantic ranking, and vector search, making it a common retrieval layer for grounded generative AI applications.

---

9. Feedback System

9.1 Design Principle

Feedback should feel intelligent without requiring an AI model during gameplay.

v0.0.1 should use a deterministic lookup chain:

1. Resolve guess to known concept.
2. Check if guess matches the answer.
3. Check puzzle-specific feedback.
4. Check concept relationship feedback.
5. Check category/subcategory fallback.
6. Return generic fallback.

9.2 Feedback Lookup Order

```
User submits guess
        ↓
Normalize guess / alias
        ↓
Resolve to concept_id
        ↓
Is concept_id correct?
        ↓
Check puzzle-specific feedback
        ↓
Check concept relationship
        ↓
Check category/subcategory
        ↓
Return generic fallback
```

9.3 Puzzle-Specific Feedback

Puzzle-specific feedback should be used for likely wrong guesses.

Example:

```
{
  "puzzleId": "ai-search-rag-001",
  "answerId": "azure-ai-search",
  "specificFeedback": {
    "cosmos-db": "Not quite. Cosmos DB stores operational application data,
while Azure AI Search creates indexes for keyword, semantic, and vector
```

```
retrieval.",
    "azure-openai": "Close. Azure OpenAI can generate responses, but Azure AI
Search retrieves the grounding data.",
    "document-intelligence": "Related. Document Intelligence extracts text and
structure from documents; Azure AI Search indexes content for retrieval."
  }
}
```

9.4 Relationship-Based Feedback

Concept relationships should provide reusable feedback when custom puzzle feedback is unavailable.

Relationship examples:

- `used_with`
- `commonly_confused_with`
- `parent_of`
- `child_of`
- `alternative_to`
- `stores_data_for`
- `triggers`
- `monitors`
- `secures`
- `integrates_with`
- `too_broad_for`
- `too_specific_for`
- `same_category`

Example relationship:

```
{
  "fromConceptId": "azure-openai",
  "toConceptId": "azure-ai-search",
  "relationshipType": "used_with",
  "strength": 4,
  "explanation": "Azure OpenAI and Azure AI Search are often used together in
RAG applications, but Azure OpenAI generates responses while Azure AI Search
retrieves grounding data."
}
```

9.5 Category-Based Feedback

If no custom or relationship feedback exists, use metadata.

Example:

> You are in the right general domain. Cosmos DB is mainly used for globally distributed operational data storage, but this clue points toward indexing and retrieving content for search and RAG.

9.6 Generic Fallback

If the guess and answer have no useful relationship:

> Not quite. This concept solves a different kind of problem than the one described in the clue.

---

10. Content Strategy

10.1 Source of Truth for v0.0.1

For the POC, Git should be the source of truth.

Content should live in versioned files:

```
/content
  /certifications
    az-204.json
    ai-102.json
    claude-dev.json
  /concepts
    azure.json
    anthropic.json
  /relationships
    azure-relationships.json
    anthropic-relationships.json
  /puzzles
    az-204-monitoring-pack.json
    ai-102-rag-pack.json
```

10.2 Why Git for POC

Git-backed content is preferred for v0.0.1 because it is:

- Cheap
- Version controlled
- Easy to review through pull requests
- Easy to seed into a database later
- Friendly to manual editing
- Suitable for mostly static educational content

10.3 Future Production Direction

In production, content should eventually move into a database-backed CMS/admin workflow.

Recommended progression:

1. **v0.0.1:** Git-backed static content
2. **v0.1.x:** Git-backed content seeded into Postgres
3. **v0.2.x:** Admin dashboard edits content in database
4. **Later:** Database becomes source of truth, with export/versioning support

10.4 Content Authoring Workflow

v0.0.1 content should be manually reviewed.

Recommended workflow:

1. Create concept list.
2. Add aliases.
3. Add categories/subcategories.
4. Add primary use descriptions.
5. Add relationships between commonly confused concepts.
6. Create puzzle prompts.
7. Add custom feedback for likely wrong guesses.
8. Review for accuracy.
9. Commit to Git.
10. Deploy.

Optional offline AI can assist with draft generation, but generated content should be reviewed before publishing.

---

11. Static Content Data Model

11.1 Certification

```
type Certification = {
  id: string;
  name: string;
  provider: string;
  level?: string;
  slug: string;
  domains: Domain[];
};
```

Example:

```
{
  "id": "az-204",
  "name": "Azure Developer Associate",
  "provider": "Microsoft",
  "level": "Associate",
  "slug": "az-204"
}
```

11.2 Domain

```
type Domain = {
  id: string;
  certificationId: string;
  name: string;
  sortOrder: number;
};
```

Example:

```
{
  "id": "monitor-troubleshoot",
  "certificationId": "az-204",
  "name": "Monitor, troubleshoot, and optimize Azure solutions",
  "sortOrder": 5
}
```

11.3 Concept

```
type Concept = {
  id: string;
  displayName: string;
  provider?: string;
  category: string;
  subcategory?: string;
  primaryUse: string;
  description?: string;
  certs: string[];
  aliases: string[];
  tags: string[];
  metadata?: Record<string, unknown>;
};
```

Example:

```
{
  "id": "azure-ai-search",
  "displayName": "Azure AI Search",
  "provider": "Microsoft",
  "category": "AI",
  "subcategory": "Search and retrieval",
  "primaryUse": "indexing and retrieving content for search, semantic ranking,
vector search, and RAG",
  "description":
"A search service for building search experiences over private, heterogeneous
content.",
  "certs": ["ai-102", "az-204"],
  "aliases": [
    "Azure AI Search",
    "Azure Cognitive Search",
    "Cognitive Search",
    "Azure Search",
    "AI Search"
  ],
  "tags": ["rag", "search", "indexing", "vector-search", "semantic-search"]
}
```


11.4 Concept Relationship

```typescript
type ConceptRelationship = {
  fromConceptId: string;
  toConceptId: string;
  relationshipType: string;
  strength: number;
  explanation?: string;
};
```

Example:


```
{
  "fromConceptId": "cosmos-db",
  "toConceptId": "azure-ai-search",
  "relationshipType": "commonly_confused_with",
  "strength": 4,
  "explanation": "Cosmos DB stores operational application data, while Azure AI
Search indexes and retrieves content for search scenarios."
}
```

11.5 Puzzle

```typescript
type Puzzle = {
  id: string;
  certificationId: string;
  domainId?: string;
  answerConceptId: string;
  prompt: string;
  difficulty: "easy" | "medium" | "hard";
  maxGuesses: number;
  tags: string[];
  explanation: string;
  sourceUrl?: string;
  status: "draft" | "published" | "archived";
  publishDate?: string;
  feedback?: Record<string, PuzzleFeedback>;
};
```

Example:


```json
{
  "id": "ai-search-rag-001",
  "certificationId": "ai-102",
  "domainId": "knowledge-mining-rag",
  "answerConceptId": "azure-ai-search",
  "prompt":
"I create searchable indexes over enterprise content and can support keyword,
semantic, and vector retrieval for grounded generative AI apps.",
  "difficulty": "medium",
  "maxGuesses": 6,
  "tags": ["rag", "search", "vector-search"],
  "explanation": "Azure AI Search creates indexes over content and supports
retrieval patterns commonly used in search and RAG applications.",
  "status": "published",
  "publishDate": "2026-07-01",
  "feedback": {
    "cosmos-db": {
      "message": "Not quite. Cosmos DB stores operational application data,
while Azure AI Search creates indexes for keyword, semantic, and vector
retrieval.",
      "closeness": 2
    },
    "azure-openai": {
      "message": "Close. Azure OpenAI can generate responses, but Azure AI
Search retrieves the grounding data.",
      "closeness": 4
```

```
    }
  }
}
```

---

12. Runtime Data Model

For v0.0.1, runtime persistence should be minimal.

12.1 Persisted Data

Store:

- User completed quiz result
- User streak/progress summary
- Aggregate puzzle stats
- Aggregate guess stats

Do not store:

- Every individual guess permanently
- Runtime AI interpretations
- Full session telemetry unless needed for debugging
- Sensitive user study data beyond basic progress

12.2 User Puzzle Result

```ts
type UserPuzzleResult = {
  userId: string;
  puzzleId: string;
  certificationId: string;
  solved: boolean;
  guessesUsed: number;
  completedAt: string;
};
```

Equivalent SQL shape:

```sql
create table user_puzzle_results (
  user_id uuid not null,
  puzzle_id text not null,
  certification_id text not null,
  solved boolean not null,
  guesses_used int not null,
  completed_at timestamptz default now(),
```

```
  primary key (user_id, puzzle_id)
);
```

12.3 User Stats

```
type UserStats = {
  userId: string;
  certificationId: string;
  puzzlesCompleted: number;
  puzzlesSolved: number;
  currentStreak: number;
  longestStreak: number;
  averageGuesses: number;
  lastCompletedPuzzleDate?: string;
};
```

Equivalent SQL shape:

```
create table user_stats (
  user_id uuid not null,
  certification_id text not null,
  puzzles_completed int default 0,
  puzzles_solved int default 0,
  current_streak int default 0,
  longest_streak int default 0,
  average_guesses numeric(5,2),
  last_completed_puzzle_date date,
  primary key (user_id, certification_id)
);
```

12.4 Aggregate Puzzle Stats

```
type PuzzleStats = {
  puzzleId: string;
  certificationId: string;
  totalCompletions: number;
  totalSolves: number;
  totalFailures: number;
  averageGuesses: number;
  solveRate: number;
};
```

Equivalent SQL shape:

```
create table puzzle_stats (
  puzzle_id text primary key,
  certification_id text not null,
  total_completions int default 0,
  total_solves int default 0,
  total_failures int default 0,
  average_guesses numeric(5,2),
  solve_rate numeric(5,2)
);
```

12.5 Aggregate Guess Stats

Instead of storing every raw guess per user, v0.0.1 should store aggregate counts.

```
type PuzzleGuessStats = {
  puzzleId: string;
  guessConceptId: string;
  guessCount: number;
  correctCount: number;
};
```

Equivalent SQL shape:

```
create table puzzle_guess_stats (
  puzzle_id text not null,
  guess_concept_id text not null,
  guess_count int default 0,
  correct_count int default 0,
  primary key (puzzle_id, guess_concept_id)
);
```

This supports analytics like:

- Most common wrong guesses
- Concepts users confuse most
- Hardest puzzles
- Average guesses per puzzle
- Puzzles that need better clues

without storing every attempt permanently.

---

13. Technical Architecture

13.1 v0.0.1 Architecture

Recommended stack:

```
Frontend:
Next.js / React

Backend:
Next.js API routes, Hono, Fastify, or .NET minimal API

Static Content:
Git-backed JSON files

Database:
Postgres for users/results/stats only

AI:
No runtime AI
Optional offline AI-assisted content drafting

Hosting:
Vercel, Cloudflare Pages, Azure Static Web Apps, or similar
```

13.2 Recommended Runtime Flow

```
Page load
  ↓
Load today’s puzzle from static content
  ↓
Load valid concept list for selected certification
  ↓
User submits guess
  ↓
Resolve guess to concept_id
  ↓
Run deterministic feedback lookup
  ↓
Return feedback
  ↓
On completion, save final result and aggregate stats
```

13.3 API Endpoints

`GET /api/certifications`

Returns available certification tracks.

`GET /api/puzzles/today?cert=az-204`

Returns today's puzzle metadata and prompt.

Should not expose the answer directly to the client unless gameplay is fully client-side and cheating is not a concern for the POC.

`GET /api/concepts?cert=az-204`

Returns valid guessable concepts for autocomplete.

Should include:

- concept ID
- display name
- aliases if needed
- category
- maybe short description

`POST /api/puzzles/:puzzleId/guess`

Request:

```
{
  "guessConceptId": "cosmos-db",
  "currentGuessNumber": 2
}
```

Response:

```
{
  "status": "incorrect",
  "isCorrect": false,
  "message": "Not quite. Cosmos DB stores operational application data, while
Azure AI Search creates indexes for keyword, semantic, and vector retrieval.",
  "guessesRemaining": 4,
  "closeness": 2
}
```

`POST /api/puzzles/:puzzleId/complete`

Called when the user solves or fails a puzzle.

Request:

```
{
  "solved": true,
  "guessesUsed": 4,
  "guessConceptIds": [
    "cosmos-db",
    "azure-openai",
    "document-intelligence",
    "azure-ai-search"
  ]
}
```

Server should:

- Save user final result if authenticated.
- Update user stats.
- Update aggregate puzzle stats.
- Update aggregate guess stats.
- Return reveal summary.

`GET /api/users/me/stats`

Returns user stats by certification track.

---

14. Caching and Cost Strategy

14.1 Cost Principles

- Static content should be served cheaply.
- Runtime database writes should happen only when a quiz is completed.
- No runtime LLM calls.
- No permanent per-guess event logging in v0.0.1.
- Aggregate stats should be updated once per completed quiz.
- Daily puzzle and concept lists should be cacheable.

14.2 Cacheable Content

The following should be aggressively cacheable:

- Certification list

- Concept list by certification
- Daily puzzle prompt
- Static puzzle metadata
- Static explanation content
- Concept relationship data

14.3 Database Writes

Database writes should occur only when:

- User completes a puzzle
- User account is created
- User stats are updated
- Aggregate stats are updated

14.4 POC Hosting Cost Strategy

For v0.0.1, avoid infrastructure complexity.

Good options:

- Static hosting + serverless functions
- Postgres free/low-cost tier
- Git-backed content loaded at build time
- Edge caching for static content

The certification content itself is not expected to be large. Even several hundred concepts per certification across many certifications should remain small. User traffic and write volume will matter more than content size.

---

15. User Experience Requirements

15.1 Home Page

Home page should include:

- Product name
- Short explanation
- Certification track selector
- Start daily puzzle button
- Optional archive link
- Optional login/signup link
- Optional stats link

15.2 Puzzle Page

Puzzle page should include:

- Certification track
- Domain or topic tag
- Difficulty
- Prompt
- Guess input with autocomplete
- Guess history
- Feedback after each guess
- Guess count remaining
- Submit button
- Reveal section after completion

15.3 Guess Input

Requirements:

- Autocomplete from valid concepts
- Alias matching
- Keyboard-friendly
- Prevent invalid submissions
- Show canonical concept name after selection

15.4 Feedback Display

Feedback should be short and actionable.

Good feedback:

> Close. Azure OpenAI can generate responses, but Azure AI Search retrieves the grounding data.

Bad feedback:

> Incorrect.

Feedback should help the user understand the distinction without over-explaining before the final reveal.

15.5 Reveal Screen

Reveal screen should show:

- Correct answer
- Explanation
- Related concepts

- User result
- Aggregate result comparison if available
- Share button
- Study/source link
- Next/archive CTA

15.6 Share Result

Share format should not reveal the answer.

Example:

```
Certordle AZ-204 #12
Solved in 4/6

🟨⬛⬛🟨🟩

certordle.app
```

For v0.0.1, a simpler share format is acceptable:

```
Certordle AI-102 #3
Solved in 4/6
```

---

16. Account and Progress Strategy

16.1 Anonymous Users

Anonymous users should be able to play the daily puzzle.

For anonymous users:

- Save local progress in browser storage.
- Prevent immediate replay through local state.
- Do not require account creation.
- Allow result sharing.

16.2 Authenticated Users

Authenticated users should get:

- Cross-device stats
- Streak tracking

- Certification-specific progress
- Completed puzzle history
- Archive access later, if desired

16.3 v0.0.1 Account Scope

Authentication is optional for the POC.

Recommended v0.0.1 approach:

- Support anonymous play first.
- Add account-backed stats only if implementation cost is low.
- Save completed results only for logged-in users.
- Save anonymous completion locally.

---

17. Analytics Strategy

17.1 v0.0.1 Analytics

Track aggregate stats only.

Useful aggregate stats:

- Puzzle completions
- Puzzle solve rate
- Average guesses used
- Most common wrong guesses
- Failure rate
- Completion rate by certification
- Completion rate by difficulty

17.2 Product Questions to Answer

The POC should help answer:

- Are users completing puzzles?
- Are clues too easy or too hard?
- Which concepts are most commonly confused?
- Which certification track gets the most engagement?
- Does deterministic feedback feel useful?
- Does the daily format encourage repeat usage?

---

18. Content Quality Guidelines

18.1 Do Not Copy Actual Exam Questions

Certordle should not recreate real certification exam questions.

Content should be based on:

- Public study guides
- Public documentation
- Public product behavior
- Original scenarios
- Original explanations

18.2 Good Puzzle Characteristics

A good puzzle should be:

- Specific enough to have one best answer
- Short enough for daily play
- Connected to certification-relevant knowledge
- Differentiated from likely distractors
- Educational after reveal

18.3 Bad Puzzle Characteristics

Avoid prompts that are:

- Too broad
- Ambiguous between multiple correct services
- Pure trivia
- Dependent on obscure pricing details
- Based on deprecated product names without alias handling
- Too similar to actual exam questions
- Impossible without memorizing exact documentation wording

18.4 Example Good Prompt

> I expose APIs to internal or external consumers, apply policies such as rate limits, and can provide a developer portal.

Answer: Azure API Management

18.5 Example Bad Prompt

> What Azure service manages APIs?

Problem: Too broad and too easy.

---

19. Future Database-Backed Production Model

Although Git is the source of truth for v0.0.1, production should eventually use database-backed content.

19.1 Future Tables

Future content tables may include:

- `certifications`
- `domains`
- `concepts`
- `concept_aliases`
- `certification_concepts`
- `concept_relationships`
- `puzzles`
- `puzzle_feedback`
- `puzzle_publish_schedule`
- `content_reviews`
- `source_references`

Runtime tables may include:

- `users`
- `user_puzzle_results`
- `user_stats`
- `puzzle_stats`
- `puzzle_guess_stats`
- `subscriptions`
- `feature_flags`

19.2 Migration Path

The POC content files should be shaped so they can later seed a database.

Recommended approach:

- Keep stable concept IDs.
- Keep stable puzzle IDs.
- Keep alias lists normalized.
- Keep certification IDs consistent.
- Keep relationships explicit.
- Avoid relying on file path as the only identifier.

---

20. Security and Integrity

20.1 POC Cheating Concerns

For v0.0.1, cheating prevention is not a major priority.

However:

- Do not expose the correct answer in obvious client-side payloads if avoidable.
- Store puzzle state server-side or validate completion server-side if accounts/stats matter.
- Use local storage for anonymous completion state.
- Prevent duplicate completion submissions for authenticated users.

20.2 Data Privacy

v0.0.1 should avoid collecting unnecessary personal data.

Store:

- User ID
- Completion status
- Certification stats
- Aggregate guess stats

Avoid:

- Full raw session logs
- Keystroke logging
- Unnecessary personal profile fields
- Sensitive study behavior beyond basic progress

---

21. Accessibility Requirements

v0.0.1 should support:

- Keyboard navigation
- Screen-reader-friendly input labels
- Sufficient color contrast
- Text feedback in addition to color feedback
- Clear focus states
- Mobile-friendly layout

Do not rely on color alone to communicate correctness or closeness.

---

22. Open Questions

1. What should the final name be: Certordle, Cloudle, CertGuess, or something else?
2. Should the first track be AZ-204 or AI-102?
3. Should users get 5, 6, or unlimited guesses in practice mode?
4. Should the clue be one line only, or should scenario clues be allowed?
5. Should a daily puzzle be global, user-specific, or certification-specific?
6. Should archive access be free from the start?
7. Should accounts exist in v0.0.1 or wait until v0.1?
8. Should aggregate guess stats include anonymous completions?
9. Should “practice mode” exist separately from the daily puzzle?
10. Should source links appear before or only after completion?

---

23. Recommended v0.0.1 Scope

Must Have

- Certification track selector
- Git-backed concept content
- Git-backed puzzle content
- Daily puzzle page
- Prompt-based guessing
- Autocomplete from answer bank
- Alias resolution
- Deterministic feedback
- Puzzle-specific feedback for common wrong guesses
- Relationship/category fallback feedback
- Completion reveal
- Local anonymous completion state
- Aggregate puzzle stats after completion
- Aggregate guess stats after completion

Should Have

- Share result
- Archive route structure
- Difficulty labels
- Related concepts on reveal
- Basic user stats model
- Clean JSON schema validation
- Content linting script

Could Have

- Authentication
- Cross-device streaks

- Admin-only content preview
- AI-assisted offline puzzle generation
- Fuzzy alias matching
- Practice mode
- Cert readiness score

Won’t Have in v0.0.1

- Runtime LLM feedback
- Paid subscriptions
- Admin dashboard
- User-generated content
- Full analytics dashboard
- Spaced repetition
- Mobile app

---

24. Suggested Milestones

Milestone 1: Content Model

- Define JSON schemas.
- Create 50–100 Azure concepts.
- Create aliases.
- Create 20–50 relationships.
- Create 10 sample puzzles.

Milestone 2: Gameplay Prototype

- Build puzzle page.
- Build autocomplete.
- Implement guess submission.
- Implement deterministic feedback.
- Implement reveal screen.

Milestone 3: Persistence

- Add local anonymous completion state.
- Add Postgres result tables.
- Save completed results.
- Update aggregate stats.

Milestone 4: Polish

- Add share result.
- Add archive structure.
- Add basic stats page.
- Add content validation script.

- Add mobile/accessibility polish.

---

25. Success Metrics

v0.0.1 should be considered successful if:

- Users can complete a daily puzzle without confusion.
- Feedback feels helpful despite no runtime AI.
- Content can be added through Git without database changes.
- The app supports at least one complete certification track sample.
- Aggregate stats show common wrong guesses.
- The system can support future migration to a production database.
- The cost to operate remains minimal.

Early product metrics:

- Puzzle completion rate
- Solve rate
- Average guesses used
- Return rate
- Share rate
- Most common wrong guesses
- Anonymous-to-account conversion rate, if accounts exist

---

26. Example Repository Structure

```
certordle/
  app/
    api/
      certifications/
      concepts/
      puzzles/
      users/
    puzzle/
    archive/
    stats/
  content/
    certifications/
      az-204.json
      ai-102.json
    concepts/
      azure.json
      anthropic.json
    relationships/
```

```
      azure-relationships.json
    puzzles/
      az-204-monitoring-pack.json
      ai-102-rag-pack.json
  lib/
    content/
      loadContent.ts
      validateContent.ts
      resolveConcept.ts
    gameplay/
      getFeedback.ts
      scoreGuess.ts
      completePuzzle.ts
    db/
      client.ts
      schema.sql
  scripts/
    validate-content.ts
    seed-db.ts
  tests/
    feedback.test.ts
    concept-resolution.test.ts
```

---

27. Example Feedback Function

```ts
type FeedbackResult = {
  status: "correct" | "incorrect" | "invalid";
  isCorrect: boolean;
  message: string;
  closeness?: number;
};

function getFeedback({
  puzzle,
  guess,
  answer,
  relationships
}: {
  puzzle: Puzzle;
  guess: Concept | null;
  answer: Concept;
  relationships: ConceptRelationship[];
}): FeedbackResult {
  if (!guess) {
```

```
    return {
      status: "invalid",
      isCorrect: false,
      message:
"I do not recognize that as a valid concept in this certification set."
    };
  }

  if (guess.id === answer.id) {
    return {
      status: "correct",
      isCorrect: true,
      message: "Correct."
    };
  }

  const customFeedback = puzzle.feedback?.[guess.id];

  if (customFeedback) {
    return {
      status: "incorrect",
      isCorrect: false,
      message: customFeedback.message,
      closeness: customFeedback.closeness
    };
  }

  const relationship = relationships.find(
    r => r.fromConceptId === guess.id && r.toConceptId === answer.id
  );

  if (relationship?.explanation) {
    return {
      status: "incorrect",
      isCorrect: false,
      message: relationship.explanation,
      closeness: relationship.strength
    };
  }

  if (guess.category === answer.category) {
    return {
      status: "incorrect",
      isCorrect: false,
      message: `You are in the right general domain. ${guess.displayName} is
mainly used for ${guess.primaryUse}, but this clue points toward $
{answer.primaryUse}.`,
      closeness: 2
```

```
    };
  }

  return {
    status: "incorrect",
    isCorrect: false,
    message: "Not quite. This concept solves a different kind of problem than
the one described in the clue.",
    closeness: 1
  };
}
```

---

28. v0.0.1 Product Decision Summary

For v0.0.1, Certordle should be built as a lightweight, deterministic, certification-focused guessing game.

Key decisions:

- Use Git as the source of truth for certification content.
- Use Postgres only for user results and aggregate stats.
- Avoid runtime AI calls.
- Use constrained autocomplete guesses.
- Resolve guesses to known concept IDs.
- Provide deterministic feedback through custom puzzle feedback, concept relationships, and metadata fallback.
- Save only completed user results, not every individual guess.
- Store aggregate guess stats to identify confusing concepts.
- Design content files so they can later migrate into a production database.

The POC should prove that the game loop is fun, useful, and educational before investing in a full content CMS, admin dashboard, or personalization engine.